VOLUME 3

COMPLETE VOLUME — CHAPTERS 3.1–3.9

Technical Architecture

The Engineering Foundation Everything Else Is Built On

Document Title	Volume 3 — Technical Architecture (Combined)
Volume Reference	Volume 3
Chapters Included	3.1 through 3.9 (9 chapters)
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Governed By	Volume 0 — Project Foundation; grounded in Volume 1 — Product Planning and Volume 2 — Product Design

Table of Contents

This volume designs the engineering foundation the Human Archive app is built on — the concrete technology choices, the reasoning behind every non-obvious one, and the shape of the codebase itself, before any implementation code is written (Volume 0's rule that no code is scaffolded before Volume 6 is approved). It contains the following 9 chapters:

VOLUME 3 — TECHNICAL ARCHITECTURE

CHAPTER 3.1

Technology Stack

The Concrete Tools the Engineering Foundation Is Built On

Document Title	Technology Stack
Chapter Reference	3.1
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Governed By	Volume 0 — Project Constitution, Section 3 (Technical Foundations)

1. Purpose & Ground Rules

Volume 0's Project Constitution already fixed four decisions as non-negotiable: Flutter/Dart as the single codebase, a Windows-first development workflow with Android as the primary test target, AWS S3 as the system of record for uploaded chunks, and offline-first as a core architectural principle. This chapter builds the rest of the stack on top of those fixed points. Every choice below that has more than one reasonable option is backed by a full Architecture Decision Record in Chapter 3.2 — this chapter states what was chosen; Chapter 3.2 states why.

A small number of decisions are intentionally left open here because Volume 0 already assigned them to a later volume: the exact backend compute/auth split (Volume 4 — Backend Architecture), the precise local database schema (Volume 6.5), and the CI/iOS build service configuration (Volume 7 — Development Environment). This chapter names the category and the leading candidate so downstream volumes have a starting point, without re-deciding what Volume 0 already deferred.

2. Mobile Application Stack

Layer	Choice	Why
Language & Framework	Flutter (stable channel) / Dart	Fixed by Volume 0, Chapter 0.2 — single codebase for Android + iOS, not open for reconsideration.
State Management	Riverpod (flutter_riverpod + riverpod_generator)	Compile-safe dependency graph, first-class async/stream support (needed for live upload-queue and checklist state), and testable without a widget tree. Full comparison in ADR-001 (Chapter 3.2).
Local Persistence (structured data)	Drift (SQLite under the hood)	Type-safe SQL, reactive queries (a Stream per query — a natural fit for Riverpod), and a real migration story as the schema grows across Projects/Tasks/Sessions/Chunks/Metadata. Full comparison in ADR-002.
Local Persistence (raw video)	Device filesystem (app-sandboxed directory), referenced by path from the Drift chunk record	Video files are large and streamed to disk during capture — they do not belong inside a SQL database; only their path, size, and checksum live in Drift (FR-META-06).
Camera & Capture	Flutter's official camera plugin, with a platform-channel extension for ultra-wide lens selection	Covers standard capture; the 0.5x/0.6x wide-angle lens selection (FR-META-03) is a device-native capability the stock plugin does not fully expose on all devices — flagged as an open engineering risk carried into Volume 6, not resolved here.
Background Upload — Android	A foreground service (flutter_foreground_task) driving Dio multipart uploads, with a persistent, dismissible-only-on-completion notification	Android increasingly restricts background execution; a large video upload needs a guaranteed-running foreground service, not a best-effort background task. Full comparison in ADR-003.
Background Upload — iOS	URLSession background transfer via a platform-channel bridge	iOS has no equivalent to an Android foreground service; background URLSession is the only Apple-sanctioned way to keep a large upload moving after the app is backgrounded. Flagged as a secondary-priority risk per Volume 0's iOS-secondary stance.
Networking / HTTP	Dio	Interceptors (for auth token refresh and retry/backoff), native multipart/form-data support, and cancellation tokens — all required by FR-UPL-03

Layer	Choice	Why
		(resumable/multipart upload to S3). Full comparison in ADR-004.
Routing / Navigation	go_router	Declarative routing that maps directly onto Chapter 2.4's tab/stack/modal/chrome-free navigation model, with typed route parameters. Full comparison in ADR-005.
Dependency Injection	Riverpod's own provider graph (no separate DI package)	Riverpod providers already are a dependency graph; introducing a second DI mechanism (e.g. get_it) alongside it would be redundant. Documented in ADR-006.
Model Serialization	freezed + json_serializable	Immutable, value-equality models for every metadata field (FR-META-01–07) and API payload, with generated JSON (de)serialization instead of hand-written, error-prone mapping code.
Testing	flutter_test (unit/widget), mocktail (mocking), integration_test (end-to-end), golden_toolkit (visual regression against the Chapter 2.8 Design System)	Covers all four testing needs the app requires: pure logic, widget rendering, full user flows, and pixel-level design-system consistency. Full strategy in Volume 9 — Testing.
Static Analysis / Lint	flutter_lints, extended per Chapter 3.7's rule set	Enforces the Effective Dart baseline the Constitution already requires (Volume 0, Chapter 0.2).

3. Backend & Cloud Services Stack

The full backend architecture is Volume 4's responsibility. This section names only what Volume 0's Constitution already fixed or strongly indicated, so the mobile stack above has something concrete to integrate against.

Layer	Choice	Why
Object Storage	AWS S3 (resumable/multipart upload)	Fixed by Volume 0, Chapter 0.2 — the system of record for every uploaded chunk (FR-UPL-03).
Authentication	Firebase Authentication (leading candidate)	Named in Volume 0's Constitution as a likely fit alongside a custom API; final confirmation and configuration is Volume 4's decision, recorded there as its own ADR.
Push Notifications	Firebase Cloud Messaging (leading candidate)	Same status as Authentication above — supports the Phase 2 notification screens (C-15) named in Chapter 2.5.
Projects / Tasks / Sessions / Metadata API	Custom REST/GraphQL API — language/framework not yet chosen	Volume 0 explicitly defers this to Volume 4 (Backend Architecture); naming it here only as a category keeps this chapter honest about what is and isn't decided yet.
Metadata Store	Not yet chosen — candidates include a relational database, a document store, or S3 object tagging	Volume 1, Chapter 1.1 (PRD) lists this explicitly as an open question resolved in Volume 4.4/4.5 (Data Dictionary & Metadata Specification).

4. Development & CI Tooling

Layer	Choice	Why
Primary Development OS	Windows	Fixed by Volume 0, Chapter 0.2 — 90–100% of development happens here, with Android as the primary test target.

Layer	Choice	Why
IDE	Visual Studio Code with the Flutter/Dart extensions, or Android Studio	Both run natively on Windows and support Flutter's hot reload; the choice is left to individual developer preference rather than mandated, since it has no effect on the shipped app.
Version Control	Git, hosted on the platform the team already uses for the Volume 0–2 documentation workflow	No new tool introduced; consistent with how this project's documents themselves have been tracked.
CI/CD — Android & general checks	GitHub Actions (or equivalent), running on standard Linux/Windows runners	Android builds, static analysis, and unit/widget tests all run without needing macOS hardware.
CI/CD — iOS build, sign, and test	A cloud Mac / CI build service (Codemagic, Ionic Appflow, or a rented Mac-in-the-cloud)	Fixed in category by Volume 0, Chapter 0.2, since no local Mac is available; the specific vendor is chosen and configured in Volume 7 — Development Environment.

5. Stack Summary — One-Page View

- Flutter/Dart app (Windows-developed, Android-first) → Riverpod for state → Drift + filesystem for local persistence → camera plugin for capture → foreground service (Android) / background URLSession (iOS) for resilient upload → Dio for networking → go_router for navigation.
- Backend: AWS S3 (fixed) for chunk storage, Firebase (leading candidate) for auth/push, a to-be-chosen custom API and metadata store (Volume 4) for everything else.
- Tooling: Windows + VS Code/Android Studio + Git + GitHub Actions, with a cloud Mac service bridging the iOS build/sign/test gap.

6. What Remains Open

- Backend compute/auth split and language/framework — Volume 4 (Backend Architecture).
- Metadata store technology — Volume 4.4/4.5 (Data Dictionary & Metadata Specification).
- Concrete Drift schema and local database file layout — Volume 6.5 (Mobile App Architecture, Local Persistence).
- Named CI/CD vendor and pipeline configuration — Volume 7 (Development Environment).
- Ultra-wide lens selection approach per device/OS version — flagged as an engineering risk carried into Volume 6, not resolved in this document.

VOLUME 3 — TECHNICAL ARCHITECTURE

CHAPTER 3.2

Architecture Decision Records

Every Choice With More Than One Reasonable Option, and Why This One Won

Document Title	Architecture Decision Records
Chapter Reference	3.2
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Rule Applied	Volume 0, Chapter 0.2 — every decision with more than one reasonable technical option is recorded here, including the options considered and why one was chosen.

1. Purpose & Format

Each record below follows the same shape: the problem being solved (Context), the realistic alternatives (Options Considered), the choice made (Decision), and what that choice commits the team to, both good and bad (Consequences). ADRs are not revisited casually — changing one requires a deliberate amendment, following the same change-request process the Constitution defines for the Constitution itself (Volume 0, Chapter 0.2, Section 9).

2. ADR-001 — State Management Library

ADR-001 — State Management Library

Status: Accepted

Context:
The app's state includes several genuinely asynchronous, long-lived streams — live upload-queue status per chunk (FR-SES-03), a re-evaluating pre-recording checklist (FR-CHK-01–04), and session/metadata state that must survive app restarts (NFR-REL-04). The state solution needs first-class async/stream support and to be testable independent of widgets, not just a way to rebuild UI on setState.

Options Considered:

- Provider — simple and widely used, but weaker at modeling async/stream state cleanly and relies more on BuildContext.
- BLoC (flutter_bloc) — strong separation of events/state and mature tooling, but more boilerplate per feature and a steeper learning curve for a small team.
- Riverpod (flutter_riverpod + riverpod_generator) — compile-safe (no BuildContext lookup failures), first-class AsyncNotifier/StreamNotifier support, and fully testable by instantiating a ProviderContainer without a widget tree.

Decision:
Chosen: Riverpod, using code generation (riverpod_generator) for providers to keep boilerplate low and catch mistakes at compile time.

Consequences:

- Every async data source in the app (upload queue, checklist, session list) is modeled as an AsyncNotifier/StreamNotifier, giving consistent loading/data/error handling app-wide (Chapter 3.9 specifies this in full).
- The team commits to Riverpod's code-generation build step (build_runner) as part of the local dev loop — an added step, but one already common in Flutter/Dart projects using freezed.
- Testing state logic requires no widget pump — a meaningful win for a Windows-first workflow where the Android emulator/device is a comparatively heavier dependency to spin up for every test run.

3. ADR-002 — Local Persistence Engine

ADR-002 — Local Persistence Engine (Structured Data)

Status: Accepted

Context:
Offline-first is a non-negotiable principle (Volume 0, Chapter 0.2). The app must persist Projects, Tasks, Sessions, chunk records, and full metadata (FR-META-08/09) locally, survive a crash or force-close with the full queue state intact (NFR-REL-04), and support a growing, migrating schema as MVP scope expands (Chapter 1.10/1.11).

Options Considered:

- Hive — fast key-value store, but weaker for relational queries (e.g. "all chunks for this session, ordered by sequence index") and has had ongoing maintenance concerns in the Flutter ecosystem.
- Isar — fast and query-capable, but a newer project with a less certain long-term maintenance trajectory for a multi-year product.
- Drift (built on SQLite) — mature, type-safe SQL with compile-time-checked queries, native reactive Streams per query (a direct fit for Riverpod's StreamNotifier), and a well-understood migration story via schema versioning.

Decision:
Chosen: Drift, with the raw video files themselves kept on the device filesystem (not in the database) and referenced by path, size, and checksum from a Drift chunk record.

Consequences:

- Every structured query the app needs (session status, chunk-by-status counts, metadata lookups for Admin's Metadata Detail screen, A-08) is expressed as ordinary, type-checked SQL rather than ad hoc key-value scans.
- Schema migrations (adding a metadata field, adding the Admin role's tables) follow Drift's versioned migration mechanism, giving a safe, tested path to evolve the local schema as later volumes (4, 6) add detail.
- The exact table definitions are Volume 6.5's responsibility, not this chapter's — ADR-002 fixes the engine, not the schema.

4. ADR-003 — Background Upload Mechanism

ADR-003 — Background Upload Mechanism

Status: Accepted

Context:
The product's core promise is that a Collector can stop recording, immediately start another session, and trust that the previous chunk uploads in the background, resiliently, across app restarts and

network loss (FR-UPL-03, NFR-REL-02/03, NFR-AVL-02). Video files are large (multiple hundreds of MB per 10-minute chunk), which rules out anything designed for small, best-effort background tasks.

Options Considered:

- workmanager alone — good for periodic/deferred work, but Android's WorkManager execution windows are not designed for guaranteed, long-running, large-file transfers and can be deferred or killed under battery/Doze restrictions.
- A simple in-app upload triggered only while the app is foregrounded — simplest to build, but directly violates the product's own stated behavior (upload continues while the Collector records the next session) and NFR-AVL-02's 30-second resume target.
- A dedicated Android foreground service (with a persistent notification) driving the actual Dio upload calls, paired with iOS's native URLSession background transfer.

Decision:

Chosen: A foreground service on Android (via flutter_foreground_task or an equivalent platform-channel implementation) and background URLSession transfer on iOS — each platform's actual sanctioned mechanism for guaranteed background network transfer, rather than a single cross-platform shortcut that would under-deliver on one OS.

Consequences:

- The Collector sees a persistent, non-dismissible-until-complete notification during active uploads on Android — an intentional, honest signal that work is happening in the background, not a UX flaw to hide.
- iOS background transfer has real OS-imposed constraints (time budgets, does not run indefinitely without app engagement); this is documented as a carried risk into Volume 6, consistent with Volume 0's iOS-secondary-priority stance.
- Because the two platforms use genuinely different native mechanisms, this is one of the few places the app cannot be 100% shared Dart logic — a thin platform-channel layer is required, scoped and isolated in Volume 6's component design (Chapter 3.4, Section 4 names this boundary).

5. ADR-004 — HTTP / Networking Client

ADR-004 — HTTP / Networking Client

Status: Accepted

Context:

Every network call needs auth-token attachment and refresh-on-401, multipart/form-data uploads for S3's multipart API (FR-UPL-03), request cancellation (a Collector navigating away mid-upload), and retry/backoff on transient failure (NFR-REL-02).

Options Considered:

- The bare http package — minimal and dependency-light, but requires hand-rolling interceptors, multipart handling, and retry logic from scratch.

- chopper — code-generated, Retrofit-style client; clean for simple REST calls but less ergonomic for the resumable multipart upload flow this app depends on.
- Dio — interceptor chain (auth refresh, logging, retry), native multipart/form-data and upload-progress callbacks, and cancellation tokens, all built in.

Decision:

Chosen: Dio, with a dedicated interceptor for auth-token refresh and a dedicated interceptor for retry/backoff, kept as two separate, independently testable interceptors rather than one combined one.

Consequences:

- Upload progress callbacks feed directly into the Upload/Sync Status UI (Chapter 2.7, C-11) without a translation layer.
- The team takes on Dio as a direct dependency across the networking layer; this is an accepted, standard choice in the Flutter ecosystem with an active maintenance history.

6. ADR-005 — Routing / Navigation

ADR-005 — Routing / Navigation

Status: Accepted

Context:

Chapter 2.4 (Navigation Structure) already defined the shape the router must support: a persistent bottom tab bar per role, stack (drill-down) navigation within each tab, modal/full-screen overlays for checklists and forms, and one deliberately chrome-free full-screen route (the Recording Screen) that suppresses the system back gesture.

Options Considered:

- Flutter's raw Navigator 2.0 API directly — maximally flexible, but verbose and easy to get subtly wrong for nested tab/stack navigation.
- auto_route — code-generated and type-safe, a reasonable alternative, but with a smaller ecosystem overlap with Riverpod-based projects than go_router.
- go_router — declarative, URL-based routing (useful for deep links to a specific Project/Task later), typed route parameters, and first-party Flutter team support.

Decision:

Chosen: go_router, with nested routes modeling the Collector and Admin tab structures from Chapter 2.4, and the Recording Screen implemented as a route that explicitly disables the system back gesture rather than relying on convention alone.

Consequences:

- The navigation graph in code is a near-direct translation of Chapter 2.4's structure, reducing the chance of drift between the documented navigation model and the implementation.

- go_router being first-party-supported reduces long-term maintenance risk relative to a smaller community package.

7. ADR-006 — Dependency Injection Approach

ADR-006 — Dependency Injection Approach

Status: Accepted

Context:
Repositories (Auth, Projects/Tasks, Recording, Upload, Metadata) need to be constructed once and made available throughout the app, and swapped for fakes in tests.

Options Considered:

- get_it — a popular, widely used service locator, but running it alongside Riverpod's own provider graph means two separate dependency systems to reason about.
- Manual constructor injection everywhere, with no framework — fully explicit, but becomes unwieldy once the dependency graph is more than a few levels deep (as it already is: UI depends on state, state depends on repositories, repositories depend on Drift/Dio/filesystem services).
- Riverpod's own provider graph, used directly as the DI mechanism.

Decision:
Chosen: Riverpod providers double as the app's dependency injection layer — a repository is itself exposed as a provider, and any provider that needs it simply reads it, following the same pattern as state.

Consequences:

- There is exactly one dependency/composition mechanism in the app, not two — anyone reading the codebase does not need to hold both Riverpod and a separate service locator in their head at once.
- Overriding a repository with a fake in tests is a first-class Riverpod feature (ProviderScope overrides), not a bolted-on testing seam.

8. Decisions Explicitly Not Made Here

Consistent with Volume 0's own scoping, the following are named as future ADRs owned by later volumes rather than decided prematurely in this chapter:

- Backend language/framework and the Auth/API split (Firebase vs. fully custom) — Volume 4, Backend Architecture.
- Metadata store technology (relational, document store, or S3 object tagging) — Volume 4.4/4.5.
- Named CI/CD vendor for iOS build/sign/test — Volume 7, Development Environment.

VOLUME 3 — TECHNICAL ARCHITECTURE

CHAPTER 3.3

System Architecture

How the Mobile App, Backend, and Cloud Storage Fit Together

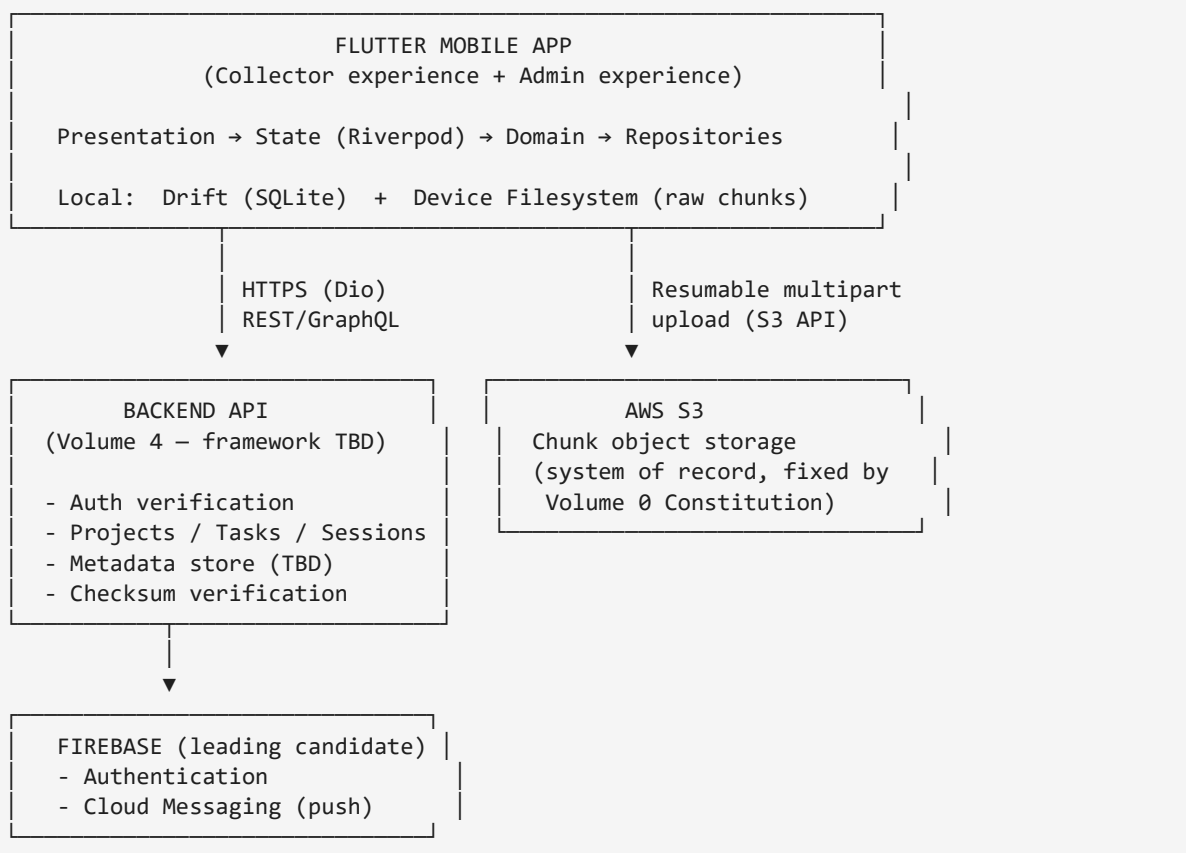
Document Title	System Architecture
Chapter Reference	3.3
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Companion Documents	Chapter 3.1 (Technology Stack), Chapter 3.2 (ADRs)

1. Purpose

Chapter 3.1 named the pieces. This chapter shows how they connect — the boundary between the Flutter app and everything it talks to over the network, and the path a single chunk of video takes from the moment it is recorded to the moment it is verified safe in cloud storage. Volume 4 (Backend Architecture) and Volume 5 (Upload & Storage Architecture) take each boundary shown here and specify it in full.

2. System Boundary Diagram (Textual)

A box-and-arrow diagram is provided visually in Volume 6's implementation documentation; the same structure is expressed here as a labeled block diagram so this document remains fully readable outside a diagramming tool.



3. The Two Roles at the System Level

Collector device: Talks to the Backend API for its assigned Projects/Tasks and to S3 directly for chunk upload; never talks to another Collector's device, and never sees data outside what BR-19 (Volume 1) permits.

Admin device: Talks to the same Backend API for Project/Task/Collector-assignment management (FR-ADM-01–06) and for reading — never writing — metadata and session status across the Projects it manages.

Both roles share one mobile codebase (Chapter 3.1) and one backend, differentiated entirely by the authenticated user's role claim, not by separate apps or separate backends (Chapter 1.6, Roles & Permissions).

4. End-to-End Data Flow — A Single Chunk's Life

This is the sequence Volume 1's requirements already specify (FR-META-08–15, FR-SES, BR-08, BR-21); this chapter locates each step in the system diagram above.

- 1. Recording: the Collector's device captures wide-angle video locally; no network involvement (checklist's network check, FR-CHK-04, only affects what happens next, not recording itself).
- 2. Chunk finalization: at the 10-minute boundary or on manual Stop, the app finalizes a chunk file on the device filesystem and immediately generates its full metadata record (FR-META-08), including a checksum (FR-META-10) — both written to Drift before anything is sent anywhere (FR-META-09).
- 3. Queueing: the chunk enters the local upload queue (Drift-backed, surviving app restarts per NFR-REL-04); if the network check found no connectivity, it sits in Queued state (pill-queued, Chapter 2.8) until connectivity returns.
- 4. Upload: the Android foreground service or iOS background transfer (ADR-003) sends the chunk to S3 via resumable/multipart upload (FR-UPL-03), while its metadata is sent to the Backend API, associated with that chunk's S3 object key (FR-META-11).
- 5. Verification: the backend independently computes or receives the checksum and compares it against the one generated at capture time (FR-META-12); only a match allows the chunk to be marked Complete.
- 6. Session completion: once every chunk in a session clears step 5, the session itself is marked Complete (FR-SES-02, BR-12) — never earlier, and never based on upload alone without metadata confirmation (BR-21).
- 7. Admin visibility: at any point in this sequence, an Admin's Session/Chunk Status view (A-07) and Metadata Detail view (A-08) reflect the current state, read directly from the backend, never from the Collector's device.

5. Cross-Cutting System Concerns

5.1 Offline-First

Every step above except upload and verification (steps 4–5) happens entirely on-device with no network dependency, per Volume 0's offline-first principle. The system is architected so that steps

1–3 can repeat indefinitely — a Collector can record many sessions back-to-back — even if steps 4–5 have not yet completed for any of them (Chapter 1.8, UC-02, Alternate Flow).

5.2 Idempotency & Exactly-Once Delivery

Because uploads are resumable and retryable (NFR-REL-02), the system must guarantee that a retried or resumed upload never creates a duplicate or corrupted object in S3 (BR-11, NFR-REL-03). This is enforced by a deterministic, collision-proof S3 object key per chunk (finalized in Volume 5.14) rather than left to chance.

5.3 Security Boundary

Locally stored video chunks are treated as sensitive until upload is confirmed (NFR-SEC-03); the Backend API is the sole arbiter of what an authenticated Collector or Admin is allowed to read or write, so no authorization logic is duplicated or trusted client-side (Chapter 1.6, Roles & Permissions; Volume 8 — Security & Compliance covers this in full).

6. What This Chapter Deliberately Defers

- Exact backend API contract (endpoints, request/response shapes) — Volume 4.
- Exact S3 bucket structure, object key format, and lifecycle policy — Volume 5.
- Exact retry/backoff timing and duplicate-prevention mechanism — Volume 5.10.
- Mobile-side implementation of the repository/service layer shown at the top of the diagram — Chapter 3.4 (Component Architecture), immediately following, and Volume 6 in full implementation detail.

VOLUME 3 — TECHNICAL ARCHITECTURE

CHAPTER 3.4

Component Architecture

The Layers Inside the Flutter App, and What Each One Is Allowed to Know

Document Title	Component Architecture
Chapter Reference	3.4
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive

Prepared With	Claude (Cowork)
Date	05 August 2026
Companion Documents	Chapter 3.3 (System Architecture), Chapter 3.9 (State Management)

1. Purpose

Chapter 3.3 drew the boundary between the app and the outside world. This chapter goes one level deeper: inside the box labeled "FLUTTER MOBILE APP," what are the layers, and what is each one allowed to depend on? The rule that makes this architecture maintainable is simple — dependencies only point inward (Presentation depends on State, State depends on Domain, Domain depends on Data), never outward, and never sideways across features without going through a shared layer.

2. The Five Layers

Layer	Contains	Responsibility
Presentation	Screens (Chapter 2.5 inventory) and reusable widgets built from the Design System (Chapter 2.8)	Renders UI and forwards user intent (taps, form input) to the State layer. Contains no business logic and no direct repository access.
State	Riverpod Notifiers/AsyncNotifiers/StreamNotifiers — one family per feature (recording, upload queue, checklist, projects/tasks, auth, admin)	Holds and exposes the current loading/data/error state for its feature (Chapter 3.9 specifies this per-feature). Talks to Domain, never directly to Drift or Dio.
Domain	Use-case classes — one class per discrete action (e.g. StartRecordingUseCase, FinalizeChunkUseCase, RunChecklistUseCase, AssignCollectorUseCase)	Encodes business rules from Volume 1 (BR-01–23) independent of any UI or storage detail. Pure Dart, no Flutter widget imports, fully unit-testable.
Data / Repositories	One repository per domain concept — RecordingRepository, UploadRepository, ProjectTaskRepository, MetadataRepository, AuthRepository	Hides where data actually comes from (local Drift cache, device filesystem, or the Backend API) behind one interface per concept — the Repository Pattern already named in Volume 0's Glossary.
Platform Services	CameraService, ChunkingService, ConnectivityService, BackgroundUploadService, SecureStorageService	Thin wrappers around plugins and platform channels (camera, foreground service/URLSession, connectivity, secure token storage). The only layer allowed to import platform-specific plugin code directly.

3. Dependency Direction (Textual Diagram)

```
Presentation  (Chapter 2.x screens, Design System widgets)
     │  reads / calls
     ▼
State  (Riverpod Notifiers — one family per feature)
     │  calls
     ▼
Domain  (Use-case classes — one class per business action)
     │  calls
     ▼
Data / Repositories  (hide local-vs-network source per concept)
     │  calls
     ▼
Platform Services  (camera, background upload, connectivity, filesystem, Drift, Dio)

Rule: an arrow only ever points downward. A Repository never imports a
Notifier; a Notifier never calls a plugin directly; a Screen never
```

```
queries Drift directly. Chapter 3.7 (Coding Standards) enforces this
with import-boundary lint rules.
```

4. Worked Example — Stopping a Recording

To make the five layers concrete, here is one real user action, Chapter 1.8's UC-01/UC-02 (Record and Upload a Session), traced through every layer:

- Presentation: the Collector taps the Stop control on the Recording Screen (C-09, Chapter 2.7).
- State: the RecordingNotifier receives the stop intent and updates its own state to "finalizing" so the UI can show the brief Local Processing state (C-10).
- Domain: FinalizeChunkUseCase runs — it is the one place that knows a chunk must be chunked, checksummed, and have its metadata generated together, atomically, per FR-META-08.
- Data / Repositories: RecordingRepository writes the chunk file reference and MetadataRepository writes the full metadata record to Drift (FR-META-09); UploadRepository then enqueues the chunk for upload.
- Platform Services: ChunkingService performed the actual file segmentation, SecureStorageService/Drift performed the actual write, and BackgroundUploadService will later perform the actual network transfer — none of which the Domain layer needed to know the mechanics of.
- The result surfaces back up: State updates to reflect the new Queued/Uploading chunk, and Presentation (C-11, Upload/Sync Status) reflects it without any layer having skipped past its neighbor.

5. Why This Shape, Specifically

Testability: Domain use-cases and Repositories are pure Dart with no widget tree dependency, so Volume 9's testing strategy can unit-test every business rule (BR-01–23) without ever booting the Flutter engine.

Isolation of platform risk: The two platform-specific risks flagged in Chapter 3.2 (ADR-003's Android/iOS background upload split, and the ultra-wide lens selection risk from Chapter 3.1) are both contained entirely inside Platform Services — if either needs a different native approach later, no other layer changes.

One repository per concept, not per screen: Multiple screens (Collector's C-11 and Admin's A-07) both read upload/session status; because that status lives behind one UploadRepository / MetadataRepository, both screens see the same truth with no risk of drift between two independently-fetched copies.

6. What This Chapter Deliberately Defers

- The concrete Dart class signatures, method names, and file locations for each layer — Volume 6 (Mobile App Architecture).
- Which specific feature modules exist and what each contains — Chapter 3.5 (Module Architecture), immediately following.
- The physical folder layout on disk — Chapter 3.6 (Folder Structure).

VOLUME 3 — TECHNICAL ARCHITECTURE

CHAPTER 3.5

Module Architecture

Dividing the Codebase by Feature, Not Just by Layer

Document Title	Module Architecture
Chapter Reference	3.5
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Companion Documents	Chapter 3.4 (Component Architecture), Chapter 3.6 (Folder Structure)

1. Purpose

Chapter 3.4 defined the five horizontal layers (Presentation → State → Domain → Data → Platform Services) that exist inside every feature. This chapter cuts the codebase the other way — vertically, by feature — so that each module contains its own Presentation/State/Domain/Data slice for one coherent area of the product, rather than one giant Presentation folder with fifty unrelated screens in it. This feature-first shape is what Chapter 3.6 turns into an actual folder tree.

2. The Modules

Module	Owns	Depends On
core	Design System primitives (Chapter 2.8, translated into Flutter widgets/theme), shared utilities, the app's Riverpod ProviderScope setup, go_router configuration, and the five-layer base classes/interfaces every other module builds on.	Nothing else in the app — every other module depends on core, never the reverse.
auth	Login (SH-02), Role Router (SH-03), session/token state, AuthRepository (Firebase Auth integration per ADR from Volume 4).	core.
onboarding	Permission priming carousel (C-01) and Permission Blocked (C-02).	core, auth (to know the authenticated role once past login).
projects_tasks	Collector's Home Dashboard, Projects List, Project Detail, Task Detail (C-03–C-06) and Admin's equivalents plus Create/Edit forms and Assign Collectors (A-01–A-06); ProjectTaskRepository.	core, auth.
recording	Pre-Recording Checklist and its failure state (C-07/C-08), the chrome-free Recording Screen (C-09), Local Processing (C-10); CameraService, ChunkingService, RunChecklistUseCase, FinalizeChunkUseCase.	core, projects_tasks (a recording session is always tied to an assigned Task).
upload	Upload/Sync Status (C-11), Session Complete (C-12), Admin's Session/Chunk Status (A-07); UploadRepository, BackgroundUploadService, the local upload queue.	core, recording (a chunk only exists once recording produces it).
metadata	Metadata Detail/Export (A-08); MetadataRepository, the full FR-META-01–07 field model (freezed classes).	core; read by both recording (to generate metadata) and upload/admin (to display and export it), but owned as its own module since metadata has independent integrity rules (BR-21/BR-22) that outlive both the recording and upload processes.
admin_shared	Admin Dashboard stat tiles (A-01) and any cross-cutting Admin-only UI not already covered by projects_tasks.	core, auth, projects_tasks.

Module	Owns	Depends On
settings	Collector/Admin Settings (C-14), account/logout.	core, auth.

3. Why Metadata Is Its Own Module, Not Part of Recording or Upload

This is the one module boundary that might look surprising, so it is worth justifying explicitly rather than leaving it implicit. Volume 1 made metadata a first-class, comprehensive requirement in its own right (the Admin/Metadata scope revision) — with its own immutability rule (BR-22), its own retention rule that outlives the raw chunk file (BR-23, FR-META-14), and its own Admin-facing surface (A-08) that is read independently of upload status. Folding metadata's code into either recording (where it is generated) or upload (where it travels alongside the chunk) would scatter a single, cohesive set of business rules across two modules. Keeping it separate means BR-21–23 live in one place, testable and auditable as a unit.

4. Module Dependency Graph

- core is the only module every other module depends on; it depends on nothing else in the app.
- auth depends only on core.
- onboarding and settings depend on core and auth.
- projects_tasks depends on core and auth.
- recording depends on core and projects_tasks.
- upload depends on core and recording.
- metadata depends on core, and is read by recording (to write metadata) and by upload/admin_shared (to display and export it) — but metadata itself depends on nothing outside core, keeping its integrity rules isolated from both.
- admin_shared depends on core, auth, and projects_tasks.

No module in this list depends on a module below it in this order, and no two feature modules depend on each other directly without going through core — this is the same inward-only dependency rule from Chapter 3.4, applied across features instead of across layers.

5. Mapping Back to Volume 1 & Volume 2

Module	Volume 1 Traceability	Volume 2 Traceability
auth	FR-ADM-07 (role enforcement), Chapter 1.6 Roles & Permissions	SH-02, SH-03
projects_tasks	FR-ADM-01–04/08, BR-18–20	C-03–C-06, A-01–A-06
recording	FR-CHK-01–05, BR-01–04	C-01, C-02, C-07–C-10
upload	FR-UPL-03/09, FR-SES, NFR-REL/AVL	C-11, C-12, A-07

Module	Volume 1 Traceability	Volume 2 Traceability
metadata	FR-META-01–15, NFR-META-01–04, BR-21–23	A-08

6. What This Chapter Deliberately Defers

- The exact folder names and file layout implementing this module list — Chapter 3.6 (Folder Structure), immediately following.
- Package/pub workspace structure (a single app vs. a Melos monorepo of packages) — Chapter 3.8 (Dependency Management) addresses this directly, since it is really a dependency-management decision, not a module-boundary one.

VOLUME 3 — TECHNICAL ARCHITECTURE

CHAPTER 3.6

Folder Structure

Turning the Module Architecture Into an Actual Directory Tree

Document Title	Folder Structure
Chapter Reference	3.6
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Companion Documents	Chapter 3.5 (Module Architecture), Chapter 3.7 (Coding Standards)

1. Purpose

This chapter is the literal, on-disk answer to Chapter 3.5: every module named there gets one top-level folder under lib/features/, and every module's internal folder mirrors the five layers from Chapter 3.4. This is the tree Volume 6 scaffolds from directly — nothing in Volume 6 should need to invent a new top-level convention that isn't already decided here.

2. Top-Level Structure

```
human_archive_app/
├── lib/
│   ├── main.dart
│   ├── app.dart                    # MaterialApp/ProviderScope/go_router wiring
│   ├── core/                       # Chapter 3.5, Section 2 — shared foundation
│   ├── features/                   # one folder per module (Chapter 3.5)
│   │   ├── auth/
│   │   ├── onboarding/
│   │   ├── projects_tasks/
│   │   ├── recording/
│   │   ├── upload/
│   │   ├── metadata/
│   │   ├── admin_shared/
│   │   └── settings/
│   └── l10n/                       # localization resources (Phase 2)
├── test/                           # mirrors lib/ 1:1 (Section 4 below)
├── integration_test/                # end-to-end flows (Volume 9)
├── assets/
│   ├── icons/
│   └── images/
├── android/                        # native project (foreground service)
├── ios/                            # native project (background transfer)
├── analysis_options.yaml           # lint rules (Chapter 3.7)
└── pubspec.yaml                    # dependencies (Chapter 3.8)
```

3. Inside One Feature Module — Worked Example (recording/)

Every feature module follows the same internal shape, mapping directly onto Chapter 3.4's five layers. recording/ is shown in full below since it is the most structurally complete example (it touches every layer, including a platform service); every other module in Chapter 3.5 follows the identical pattern.

```
features/recording/
├── presentation/
│   ├── screens/
│   │   ├── pre_recording_checklist_screen.dart      # C-07
│   │   ├── checklist_failed_screen.dart             # C-08
│   │   ├── recording_screen.dart                    # C-09
│   │   └── local_processing_screen.dart             # C-10
│   └── widgets/
│       └── checklist_item_row.dart                  # Design System §8
├── state/
│   ├── checklist_notifier.dart
│   ├── checklist_notifier.g.dart                    # riverpod_generator output
```

```
│   └── recording_notifier.dart
├── domain/
│   ├── use_cases/
│   │   ├── run_checklist_use_case.dart              # FR-CHK-01–05
│   │   └── finalize_chunk_use_case.dart             # FR-META-08
│   └── models/
│       └── checklist_result.dart                    # freezed
├── data/
│   └── recording_repository.dart
└── platform/
    ├── camera_service.dart
    └── chunking_service.dart
```

4. Test Folder — Mirrors lib/ Exactly

Volume 9 (Testing) specifies the testing strategy in full; the structural rule fixed here is that every test file lives at the identical path under test/ as the file it tests under lib/, so there is never any ambiguity about where a new feature's tests belong.

```
test/
└── features/
    └── recording/
        ├── domain/
        │   └── use_cases/
        │       └── run_checklist_use_case_test.dart
        ├── state/
        │   └── checklist_notifier_test.dart
        └── data/
            └── recording_repository_test.dart
```

5. Naming Rules for Files and Folders

- Folders: snake_case, plural for collections of like things (screens/, widgets/, use_cases/, models/), singular for a single concept's home (state/, domain/, data/, platform/).
- Files: snake_case, suffixed by what they are — _screen.dart, _notifier.dart, _use_case.dart, _repository.dart, _service.dart — so a file's role is legible from its name alone, without opening it.
- Generated files (_.g.dart from riverpod_generator/json_serializable/freezed) are never hand-edited and are excluded from code review diffs by convention, though they remain committed to version control per Volume 7's build-reproducibility rules.
- core/ internally follows the same presentation/state/domain/data/platform shape where relevant (e.g. core/theme/ for the Design System translation, core/router/ for go_router config), rather than becoming an unstructured dumping ground.

6. What This Chapter Deliberately Defers

- The actual Dart source inside each file — this document fixes structure, not implementation; no code is written before Volume 6 is approved (Volume 0, Chapter 0.2).

- Drift table/schema files' exact location within recording/data/ and metadata/data/ — Volume 6.5.
- Android/ and ios/ native project internals beyond the top-level folder shown — Volume 6 and Volume 7.

VOLUME 3 — TECHNICAL ARCHITECTURE

CHAPTER 3.7

Coding Standards

How Code Is Written, Named, Checked, and Reviewed

Document Title	Coding Standards
Chapter Reference	3.7
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Governed By	Volume 0, Chapter 0.2 (Project Constitution) — Effective Dart baseline, clean static analysis required before any code is done.

1. Purpose & Baseline

Volume 0's Constitution already fixed the two non-negotiable rules this chapter builds on: the official Effective Dart style guide is the baseline for all Dart/Flutter code, and static analysis must run clean before any code is considered done. This chapter turns those two sentences into a concrete, enforceable rule set — the actual lint configuration, naming conventions, and review checklist Volume 6's implementation work is held to.

2. Static Analysis Configuration

analysis_options.yaml (Chapter 3.6's project root) extends flutter_lints and adds the following project-specific rules, chosen to enforce the layer and module boundaries fixed in Chapters 3.4 and 3.5:

```
include: package:flutter_lints/flutter.yaml

linter:
  rules:
    - always_use_package_imports
    - avoid_print                 # use a proper logger (Section 6 below)
    - prefer_const_constructors
    - prefer_final_locals
    - unnecessary_await_in_return
    - avoid_dynamic_calls
    - public_member_api_docs      # domain/ and data/ layers only (Section 5)
```

Import-boundary enforcement (Presentation never imports Drift/Dio directly; a Repository never imports a Notifier) is not something the stock Dart linter checks natively. This project addresses it with a custom_lint / import_lint rule set, configured to fail CI (Volume 7) rather than relying on reviewer memory alone.

3. Naming Conventions

These extend, rather than replace, the naming table already fixed in Volume 0's Constitution (API endpoints kebab-case, database tables snake_case, S3 object keys per Volume 5.14). The additions below cover Dart/Flutter-specific naming not addressed there.

Element	Convention	Example
Classes, enums, extensions, typedefs	UpperCamelCase	RecordingNotifier, ChecklistResult
Files	snake_case, suffixed by role (Chapter 3.6, Section 5)	recording_notifier.dart
Variables, functions, parameters	lowerCamelCase	chunkDurationMinutes
Constants	lowerCamelCase (per modern Effective Dart, not SCREAMING_CASE)	kMaxChunkDurationMinutes for a top-level const, per Effective Dart's k-prefix convention
Riverpod providers	lowerCamelCase, suffixed Provider	uploadQueueProvider
Use-case classes	UpperCamelCase, suffixed UseCase	FinalizeChunkUseCase

Element	Convention	Example
Repository interfaces vs. implementations	Interface has no suffix; implementation is suffixed Impl	UploadRepository (interface), UploadRepositoryImpl (Drift/Dio-backed)
Requirement-ID references in code comments	The exact ID from Volume 1, unmodified	// Enforces BR-21 — see Volume 1, Chapter 1.9

4. Traceability Comments

Because this entire codebase exists to satisfy specific, numbered requirements from Volume 1 (FR-*, NFR-*, BR-*, US-*), any code implementing a rule non-obvious from its own logic alone carries a one-line comment citing the ID, as shown in the table above. This is not decorative — it means a future engineer questioning "why does this retry only twice" or "why is this field immutable" finds the answer at the point of the decision, not by searching four other volumes.

5. Documentation Comments

- Every public class and method in the domain/ and data/ layers (Chapter 3.4) carries a dartdoc comment describing its purpose and any non-obvious behavior — enforced by the public_member_api_docs lint rule, scoped to only those two layers so presentation-layer boilerplate isn't forced to over-document.
- Widgets in presentation/ are documented only where their behavior isn't obvious from the screen ID they implement (e.g. a widget implementing C-09's chrome-free override deserves a comment; a plain label-and-value row does not).

6. Logging

print() is banned by lint rule (Section 2); all diagnostic output goes through a single project-wide logger, with log levels (debug/info/warn/error) so Volume 7's CI and Volume 9's testing tooling can filter noise from signal. This is a small rule with an outsized payoff for a Windows-first workflow: as much debugging as possible should be diagnosable from structured logs rather than requiring a physical device in hand.

7. Immutability & Null Safety

- All domain models (metadata fields, checklist results, session/chunk state) are immutable freezed classes (Chapter 3.1) — no mutable model ever crosses a layer boundary, which is what makes BR-22's "system-generated metadata is immutable" rule enforceable in code, not just in policy.
- Dart's sound null safety is used throughout with no suppressions (no ! without a preceding null check already proven by control flow) — a nullable field means the business rule genuinely allows absence (e.g. GPS location, FR-META-05, which depends on permission), never a shortcut around an unhandled case.

8. Widget & State Conventions

- Every screen widget is a ConsumerWidget or ConsumerStatefulWidget (Riverpod's widget base classes) — plain StatefulWidget with manual setState is not used for anything that represents app state also visible to another screen (e.g. upload status, visible on both C-11 and A-07).
- Widgets under 3.6's widgets/ folders are kept presentation-only — no repository or use-case call originates inside a widget's build method; a widget reads a provider's already-computed state and calls back up through its Notifier.

9. Code Review Checklist

Every pull request is checked against this list before merge, in addition to whatever Volume 9's automated test suite already verifies:

- Does this change respect the layer dependency direction from Chapter 3.4 (no upward or sideways imports)?
- Does this change stay inside its module's boundary from Chapter 3.5, or does a new cross-module dependency need to be justified and documented?
- Is every non-obvious business rule in this change traceable to a Volume 1 requirement ID in a comment?
- Does static analysis run clean, with zero suppressed lint warnings introduced by this change?
- Are new or changed public APIs in domain/ or data/ documented per Section 5?
- Are tests added or updated per Volume 9's strategy for the layer being touched?

10. What This Chapter Deliberately Defers

- The full automated test strategy and coverage targets — Volume 9 (Testing).
- CI pipeline configuration that enforces Sections 2 and 9 automatically — Volume 7 (Development Environment).
- Security-specific coding rules (secret handling, secure storage) — Volume 8 (Security & Compliance).

VOLUME 3 — TECHNICAL ARCHITECTURE

CHAPTER 3.8

Dependency Management

How Packages Enter the Project, and How They're Kept Under Control

Document Title	Dependency Management
Chapter Reference	3.8
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Companion Documents	Chapter 3.1 (Technology Stack), Chapter 3.2 (ADRs)

1. Purpose

Chapter 3.1 named the packages the app is built on; each nontrivial one already has an ADR in Chapter 3.2 justifying it. This chapter is about everything after that first decision: how pubspec.yaml is structured, how a new dependency gets added later without becoming an ungoverned free-for-all, and the constraint this project cares about more than most — every dependency must actually build on Windows for the parts of the workflow that run there (Volume 0, Chapter 0.2).

2. Project Structure — Single App, Not a Monorepo

The app is a single Flutter package (Chapter 3.6's human_archive_app/), not a Melos-managed monorepo of internal packages. Chapter 3.5's module boundaries are enforced by folder convention and lint rules (Chapter 3.7), not by splitting each module into its own publishable package. A monorepo buys stronger compile-time isolation between modules, at the cost of meaningfully more tooling overhead — for a single app with one team, that trade isn't worth it yet. This is revisited only if the codebase later splits across multiple apps (e.g. a genuinely separate Admin web app, per Chapter 1.1's open question).

3. Version Pinning Policy

Dependency Type	Pinning Rule	Why
Flutter SDK	Pinned to a specific stable version in fvm (Flutter Version Management) config, not just "stable channel"	Windows-first development across possibly multiple machines/CI runners must all build against the exact same Flutter version, or subtle version-skew bugs become a Windows-specific debugging tax.
Core architectural packages (riverpod, drift, dio, go_router — Chapter 3.1)	Caret constraint on the minor version (^x.y.z), reviewed and bumped deliberately, never on an automated schedule alone	These packages are load-bearing for the whole app; an unreviewed automatic upgrade could silently change behavior in the upload/metadata path this product depends on for its core promise.
Smaller, leaf-level utility packages	Caret constraint, can be bumped opportunistically as part of unrelated PRs	Lower blast radius if a minor version introduces a small change.
Dev-only dependencies (test, lint, build_runner tooling)	Caret constraint, bumped freely	Never ships in the release binary; broken dev tooling is inconvenient, not user-facing.

4. Adding a New Dependency — The Process

Any new package proposed for pubspec.yaml goes through this checklist before it's added, not after:

- 1. Does an already-approved package (Chapter 3.1's stack) already solve this? Prefer extending existing usage over adding a new package for a marginally better fit.

- 2. Does it build cleanly as part of the Windows-first workflow (Volume 0, Chapter 0.2) for whatever portion of development happens there — i.e. does adding it break `flutter analyze` / `flutter test` on Windows, even if the package itself targets mobile? Native plugins with Windows-side build steps are checked explicitly, since this has been a known pain point in the Flutter plugin ecosystem.
- 3. Is it actively maintained (a commit or release within the last 6–12 months, no unresolved critical issues) and does it support the currently-pinned Flutter SDK version?
- 4. Is its license compatible with commercial use (MIT, BSD, Apache 2.0 are pre-approved; anything else is checked individually before adding)?
- 5. If it's a load-bearing architectural choice rather than a small utility, does it need its own ADR in Chapter 3.2 (per Volume 0's rule that any decision with more than one reasonable option gets one)?
- 6. Is it added with an explicit version constraint per Section 3's pinning policy — never a bare, unconstrained dependency?

5. Dependency Review Cadence

- `flutter pub outdated` is run and reviewed at the start of each development phase boundary (per Volume 1's roadmap phases), not continuously — batching upgrades avoids constant churn while still preventing the dependency tree from silently aging for a year or more.
- Any dependency with a published security advisory is patched immediately, outside the normal cadence, regardless of phase boundary.

6. Minimal-Surface Principle

The guiding principle behind Sections 3–5 is that every dependency is a piece of code the team didn't write but is nonetheless responsible for — for its bugs, its maintenance lifecycle, and its Windows-buildability. The stack named in Chapter 3.1 was deliberately kept to one package per real need (one state management library, one HTTP client, one router) rather than pulling in multiple overlapping packages for the same job.

7. What This Chapter Deliberately Defers

- The exact CI step that runs `flutter pub outdated` / security scanning and where its output is surfaced — Volume 7 (Development Environment).
- License-compliance recordkeeping for the shipped app (attribution files, etc.) — Volume 8 (Security & Compliance).

VOLUME 3 — TECHNICAL ARCHITECTURE

CHAPTER 3.9

State Management

How Riverpod Models Every Piece of Live State in the App

Document Title	State Management
Chapter Reference	3.9
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Companion Documents	Chapter 3.2 (ADR-001), Chapter 3.4 (Component Architecture)

1. Purpose

ADR-001 (Chapter 3.2) chose Riverpod and named the reason: first-class async/stream support and testability without a widget tree. This chapter is the deep dive — the concrete pattern every feature's state follows, so two different features never invent two different ways to represent "loading, then data, then maybe an error."

2. The One Pattern: AsyncValue Everywhere

Every piece of state that can be loading, present, or failed — which is nearly everything in this app, since almost all of it ultimately depends on local storage or network — is modeled as Riverpod's built-in AsyncValue<T>, produced by an AsyncNotifier or StreamNotifier. This is a single, consistent shape across the entire app:

```
AsyncValue<T>
  ├── AsyncLoading()            → show a loading state (Chapter 2.9, §4.1)
  ├── AsyncData(T value)        → show the real content
  └── AsyncError(Object, StackTrace)  → show a named, actionable error
                                        (Chapter 2.9, §3 — never a generic
                                         "Something went wrong")
```

Presentation-layer widgets consume this with AsyncValue's own .when(loading:, data:, error:) pattern, so the three UX states from Chapter 2.9 are structurally impossible to forget — the compiler requires all three branches to be handled.

3. State by Feature

Feature State	Provider Type	What It Holds
Checklist (C-07/C-08)	AsyncNotifier<ChecklistResult>	The live pass/fail result of FR-CHK-01–04, re-evaluated whenever a dependent value (e.g. battery level) changes while the screen is open.
Recording (C-09/C-10)	Notifier<RecordingState> (a plain, non-async state machine: idle → recording → finalizing)	Not itself an AsyncValue, since recording is a synchronous local state machine, not something that can fail from a network or storage cause the way most other state can.
Upload Queue (C-11, A-07)	StreamNotifier<List<ChunkUploadStatus>>	A live Stream sourced from a Drift reactive query, so a chunk's status pill updates in real time as its row changes in the local database, without manual polling.
Session Completion (C-12)	Derived/computed from Upload Queue state (no separate provider)	A session is Complete exactly when every one of its chunks in the Upload Queue state reaches Complete (FR-SES-02) — expressed as a derived provider, never duplicated as separately-tracked state that could drift out of sync.
Projects/Tasks (C-03–C-06, A-01–A-06)	AsyncNotifier<List<Project>> / AsyncNotifier<List<Task>>	Backend-sourced lists, cached locally via the ProjectTaskRepository so the Collector/Admin dashboards work offline against the last-synced data (Chapter 1.1 offline-first principle).
Metadata Detail (A-08)	AsyncNotifier<ChunkMetadata>	The full FR-META-01–07 field set for one chunk/session, fetched read-only; the notifier exposes no mutating method for system-generated fields, which is how BR-22's

Feature State	Provider Type	What It Holds
		immutability rule is enforced at the state layer, not just in the UI.
Auth / Session	AsyncNotifier<AuthState> (unauthenticated / authenticated-with-role / expired)	Drives the Role Router (SH-03, Chapter 2.4 §4) and is read by every other feature's repository layer to attach the current auth token.
Assign Collectors (A-06)	AsyncNotifier<List<CollectorAssignment>>	Loaded with current assignment state, mutated locally as the Admin toggles checkboxes, and only committed (FR-ADM-03/04) on explicit Confirm — matching Chapter 2.7's spec that the confirm action is disabled until a real change is made.

4. Offline Persistence of State — Not Just In-Memory

A plain in-memory Riverpod provider would lose the upload queue and any in-progress checklist state on an app kill — unacceptable given NFR-REL-04's requirement that the full queue state survive a crash or force-close. The Upload Queue and Recording state therefore both hydrate from Drift on app start (Chapter 3.4's Data layer), not from a cold, empty default — the Notifier's initial build() reads the last-persisted state before showing anything, so a StreamNotifier is never confused for the source of truth: Drift is the source of truth, and the StreamNotifier is a live view onto it.

5. Error Modeling

AsyncError alone is a generic Dart exception/stack trace — not enough to satisfy Chapter 2.9's rule that every failure names a specific cause and fix. Each feature that can fail defines its own sealed error type (a freezed union) rather than throwing a bare Exception, so the Presentation layer can pattern-match to the exact Chapter 2.9-specified message:

```
sealed class ChecklistFailure {}
class PermissionDenied extends ChecklistFailure {}
class InsufficientStorage extends ChecklistFailure { final int freeBytes; }
class BatteryTooLow extends ChecklistFailure { final int percent; }
class NoNetwork extends ChecklistFailure {}

// Presentation maps each case directly to the Chapter 2.9 copy table —
// never a single generic "Checklist failed" string.
```

6. Testing State

Because every Notifier is plain Dart reading from a Repository interface (Chapter 3.4), Volume 9's test strategy overrides the repository provider with a fake implementation via ProviderContainer, and asserts on the resulting AsyncValue sequence (loading → data, or loading → error) with no widget pump required — the same testability point already flagged in ADR-001's consequences.

7. What This Chapter Deliberately Defers

- The exact Drift table schema each Repository reads from — Volume 6.5.

- The exact retry/backoff state machine behind a failed upload chunk — Volume 5.10, reflected here only as "Failed" being one visible AsyncValue/state outcome.
- Full widget-level test cases per screen — Volume 9 (Testing).